

談話会発表 論文紹介



Data Protection in IoT using CoAP based on Enhanced DTLS

Sara Alayda, Randa Ahmed ,
Jouf University, College of Computer and Information Sciences,
Assistant professor, Jouf University, College of Computer and
Information Sciences, Saudi Arabia, Published Date: August 06,
2023 International Journal of Advanced Trends in Computer Science
and Engineering

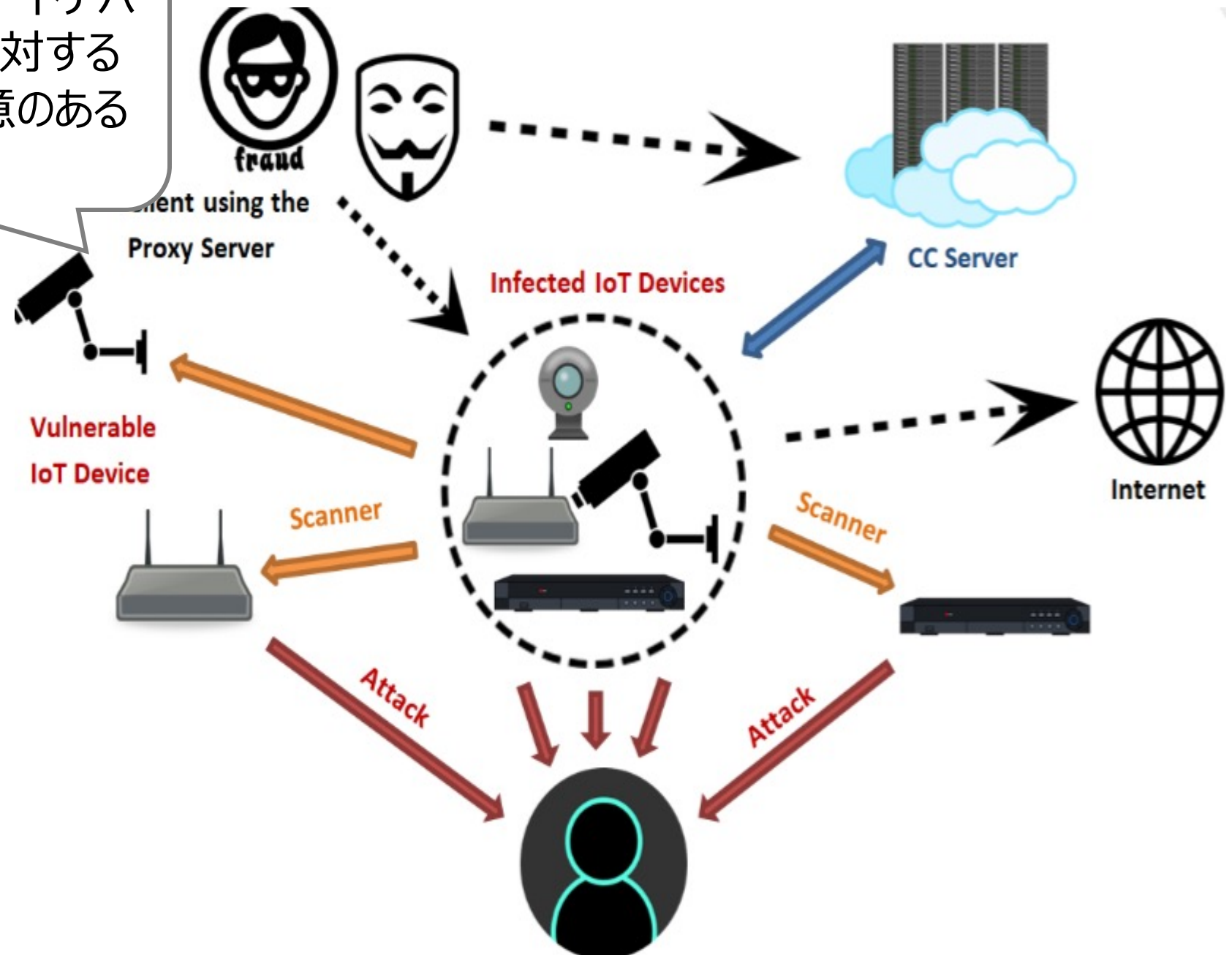
所属 : 情報基盤システム学研究室
発表者 : DAVAAKHUU UNDARMAA
指導教員 : 池部 実 講師 吉崎 弘一 教授
発表日 : 2025年6月12日
会場 : セミナー1(第三講義棟)

目次

- 研究背景・目的
- CoAPとDTLSの基礎
- Enhanced DTLSの特徴と提案手法1
- 実験方法と評価
- まとめ・今後の課題

IoT攻撃とは、スマートデバイスやネットワークに対する不正アクセスや悪意のある行為を指します。

IoT攻撃の被害には、個人情報の漏洩、ネットワークの制御奪取、サービス停止（DDoS攻撃）などが含まれます。特に、スマートホームや医療機器など重要なIoTデバイスへの攻撃は、大きな社会的影響を及ぼす可能性があります。したがって、IoTセキュリティの強化は不可欠です。



研究背景 2023年に組織が警戒すべき7つのサイバー攻撃 (Cybersecurity Infographic)

2024年に組織が警戒すべき4つのサイバー攻撃



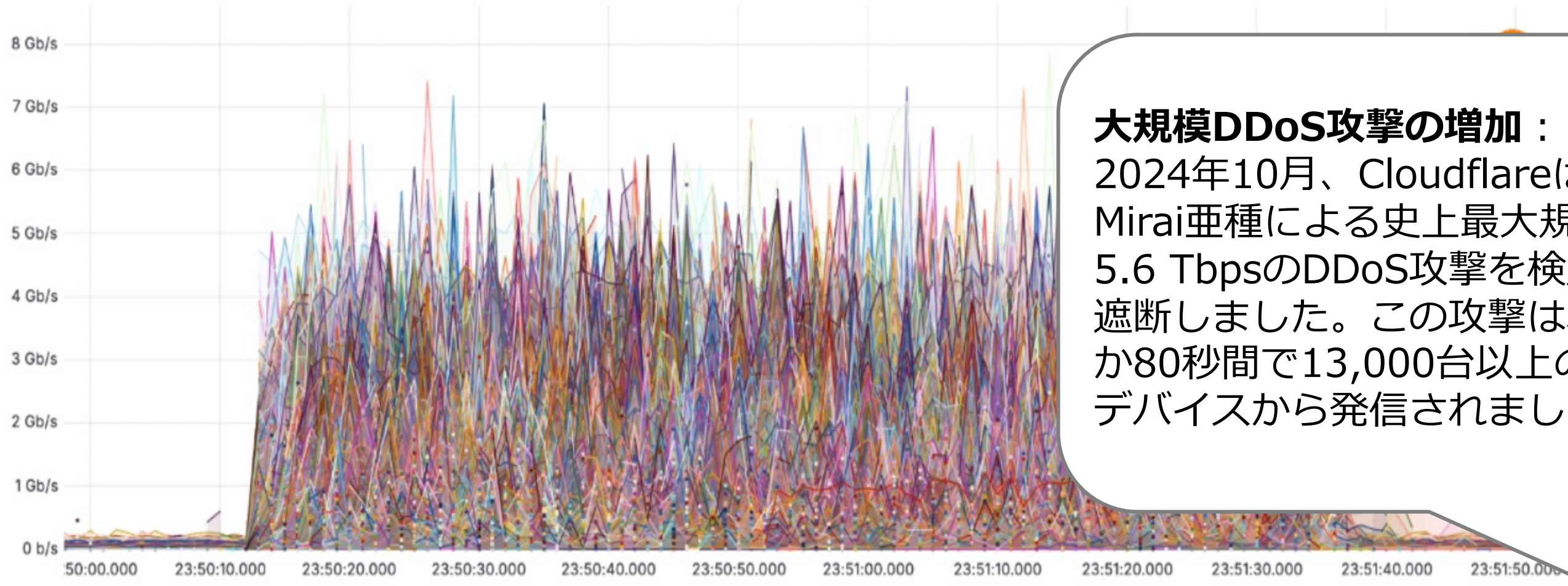
Mirai BotnetsとDDoS攻撃

Device Vulnerabilities

Man-in-the-Middle（中間者）攻撃

（Ransomware）によるIoT乗っ取り

5.6 Tbps DDoS attack delivered by an average of 5,500 unique source IPs per second



大規模DDoS攻撃の増加：
2024年10月、CloudflareはMirai亜種による史上最大規模の5.6 TbpsのDDoS攻撃を検知・遮断しました。この攻撃はわずか80秒間で13,000台以上のIoTデバイスから発信されました。

攻撃を受けやすい主なIoT機器



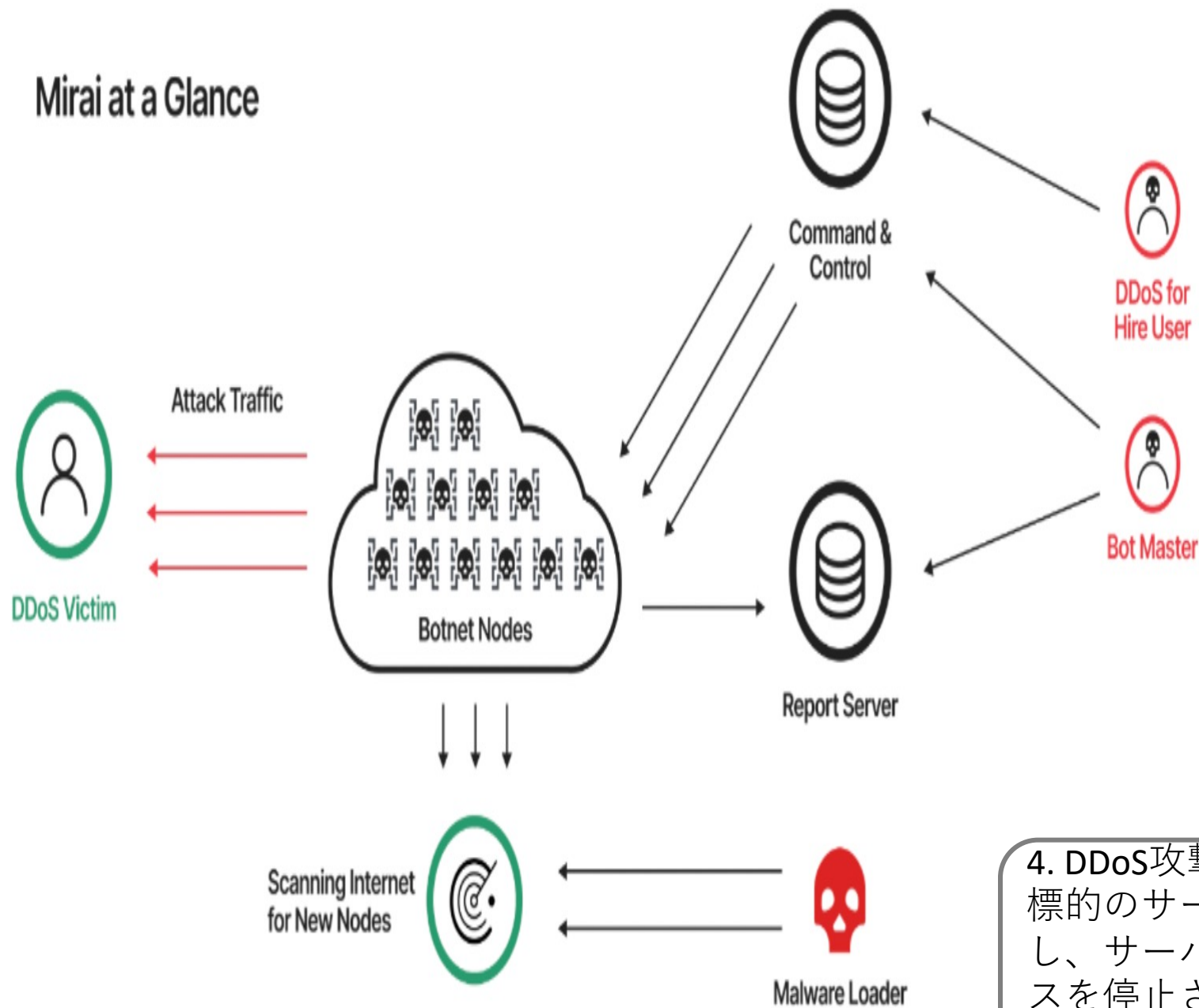
- **監視カメラ（IPカメラ）**
設定不備やパスワードの弱さにより、外部からアクセスされやすい
- **スマートルーター**
ネットワークの出入口として常に攻撃対象となる
- **Wearable Devices（Smart Watches など）**
個人情報扱うため、プライバシーリスクが高い
- **Smart Home Appliances（スマート冷蔵庫、照明など）**
セキュリティ機能が不十分な製品が多い
- **Connected Car（ネット接続された自動車）**
通信機能を悪用される可能性がある

IoT デバイスが攻撃に対して脆弱なのはなぜですか？



現在IoTで最も多い攻撃タイプ⇒ Mirai Botnet DDoS Attack

Mirai at a Glance



1. 脆弱なIoT機器の探索

インターネットに接続された機器の弱いパスワード、開放されたポート、ファームウェアの脆弱性を探し、感染を試みる。

2. 悪意のあるコードのインストール

もし機器のパスワードが簡単に推測できる場合や、ファームウェアに脆弱性がある場合、**Mirai**のコードをインストールする。

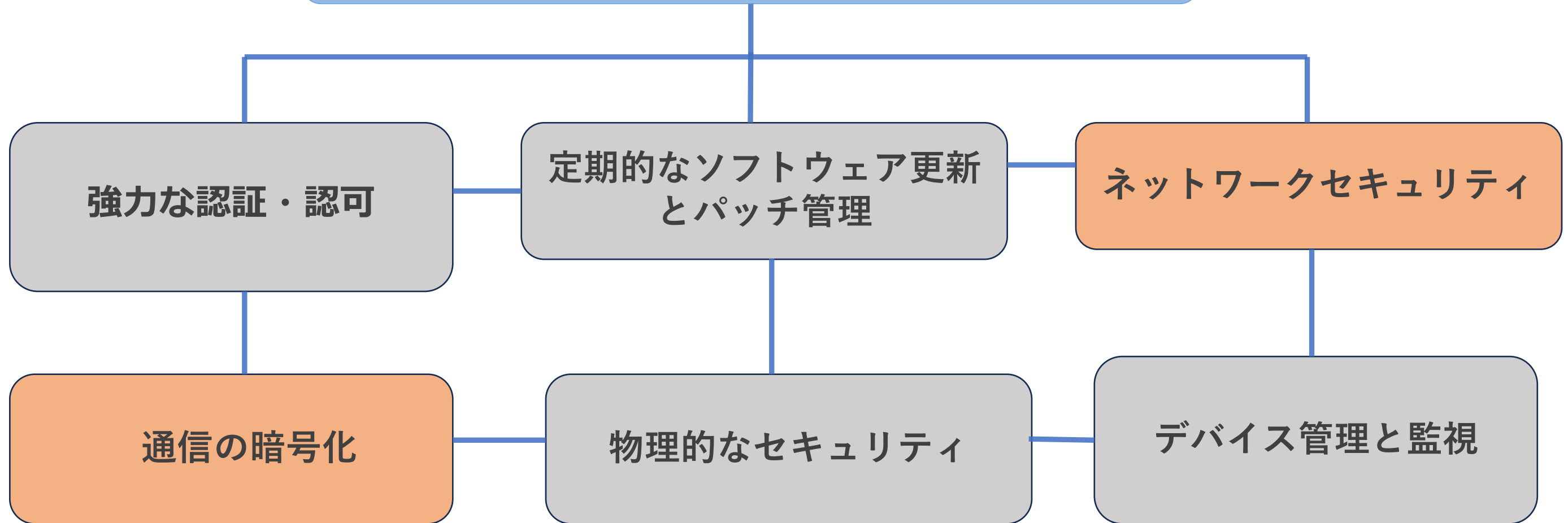
3. Botnet の形成

感染した機器は**Mirai**のコントロールサーバーと接続し、一つのボットネットとして統合される。

4. DDoS攻撃の実行

標的のサーバーに大量のトラフィックを送信し、サーバーのリソースを枯渇させ、サービスを停止させる。

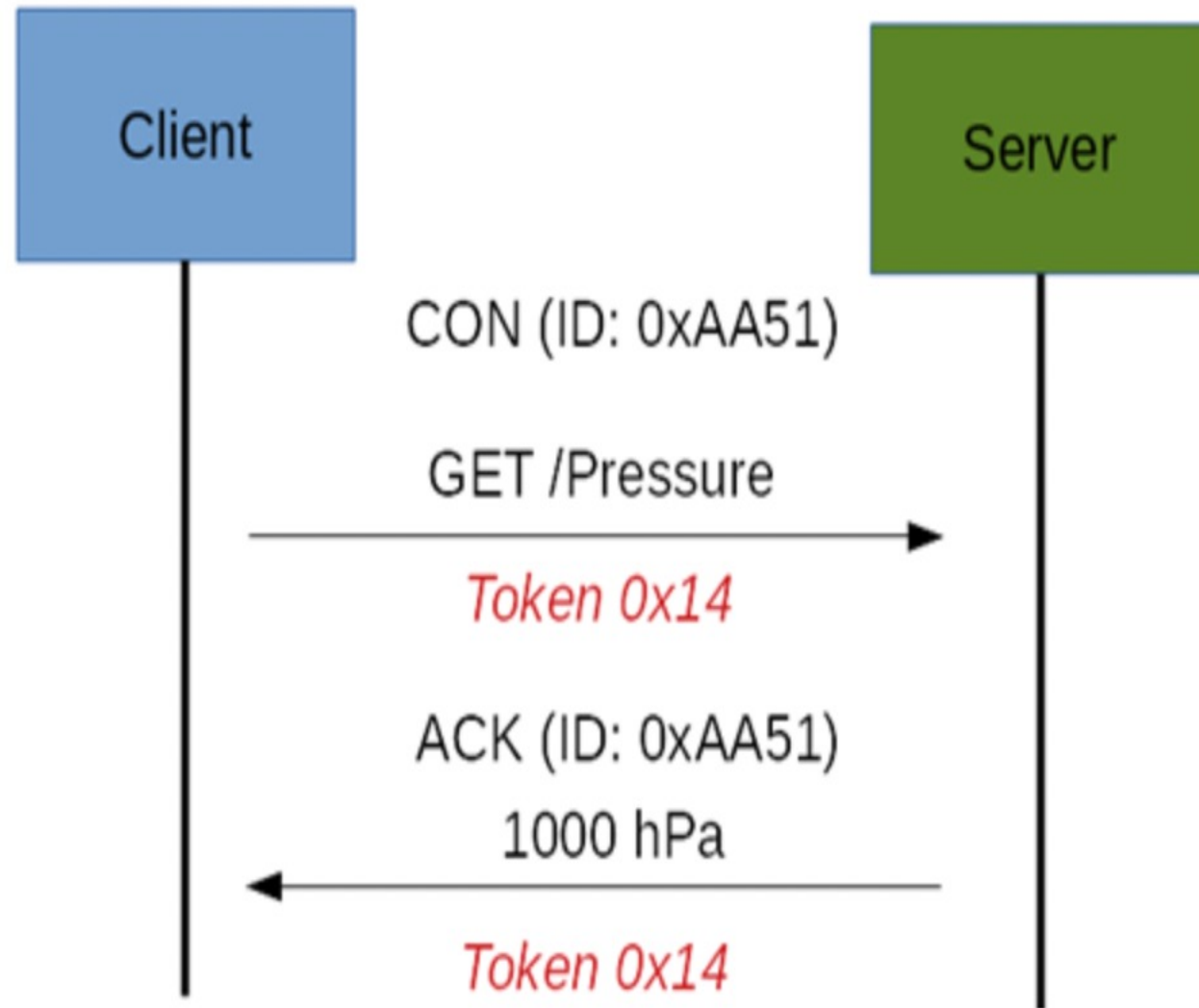
IoTのセキュリティ対策は多層的なアプローチが重要



目次

- 研究背景・目的
- CoAPとDTLSの基礎
- Enhanced DTLSの特徴と提案手法
- 実験方法と評価
- まとめ・今後の課題

• CoAP (Constrained Application Protocol) 1/2



CoAP (Constrained Application Protocol)

リソースが制限されたIoTデバイス向けに設計された軽量なアプリケーション層プロトコルであり、UDP上で動作し、HTTPに似たRESTful通信を実現します。

CoAPメソッド (GET、PUT、POST、DELETE) を使用する主な目的は、低消費電力で制約のある環境で、リソース (センサーデータ、コンフィギュレーション、デバイス設定など) に対して CRUD (Create、Read、Update、Delete) 操作を実行することです。

DTLS(Datagram Transport Layer Security) 1/2



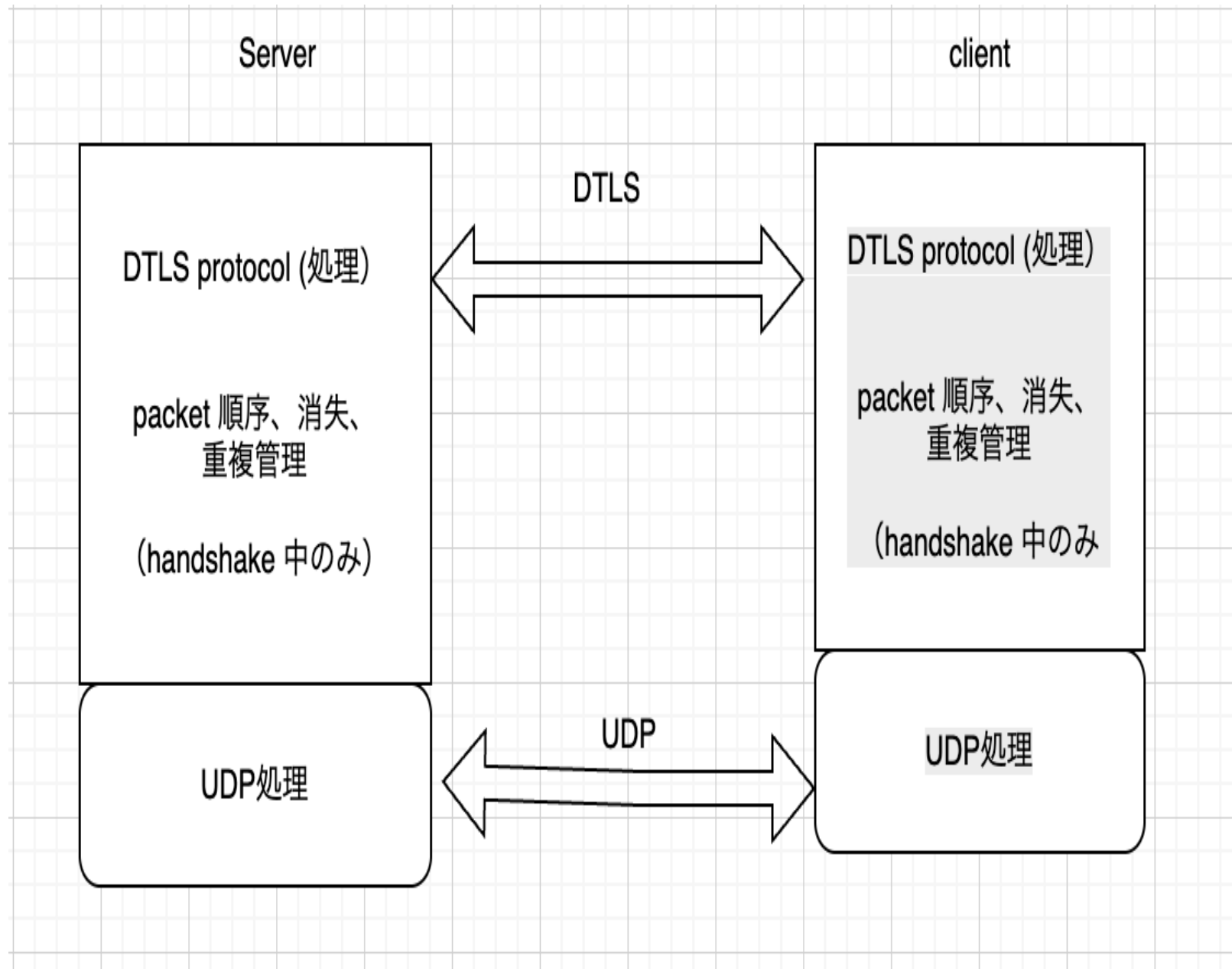
DTLSはUDPのような軽量だけど不安定な通信の上でセキュリティを実現するためのトランスポート層のプロトコルです。

- TLSのセキュリティ機能（暗号化、認証、改ざん検出）をそのままUDPに適用
- パケットの損失や順序の入れ替えに対応可能（IoT環境に最適）
- CoAPなどのUDPベースのプロトコルと組み合わせて使用される
- 軽量で高速な通信が可能のため、リソース制限のあるIoTデバイスに適している



主な特徴

DTLS(Datagram Transport Layer Security) 2/2



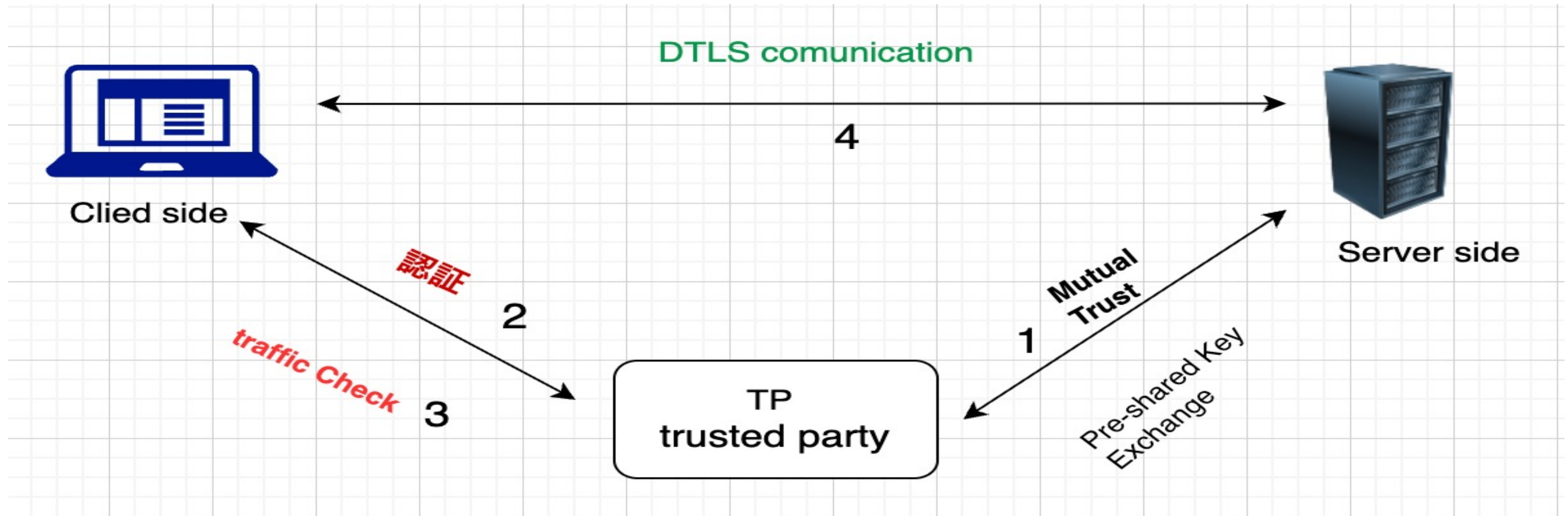
- **DTLS層**：DTLSプロトコルによる暗号化通信を処理します。
- **DTLSプロトコル処理**：暗号化・復号化、認証、メッセージの整合性確認などの処理を行います。
- **UDP層**：下層のトランスポートプロトコルで、接続レス型（connectionless）で軽量な通信を担当します。

研究の目的

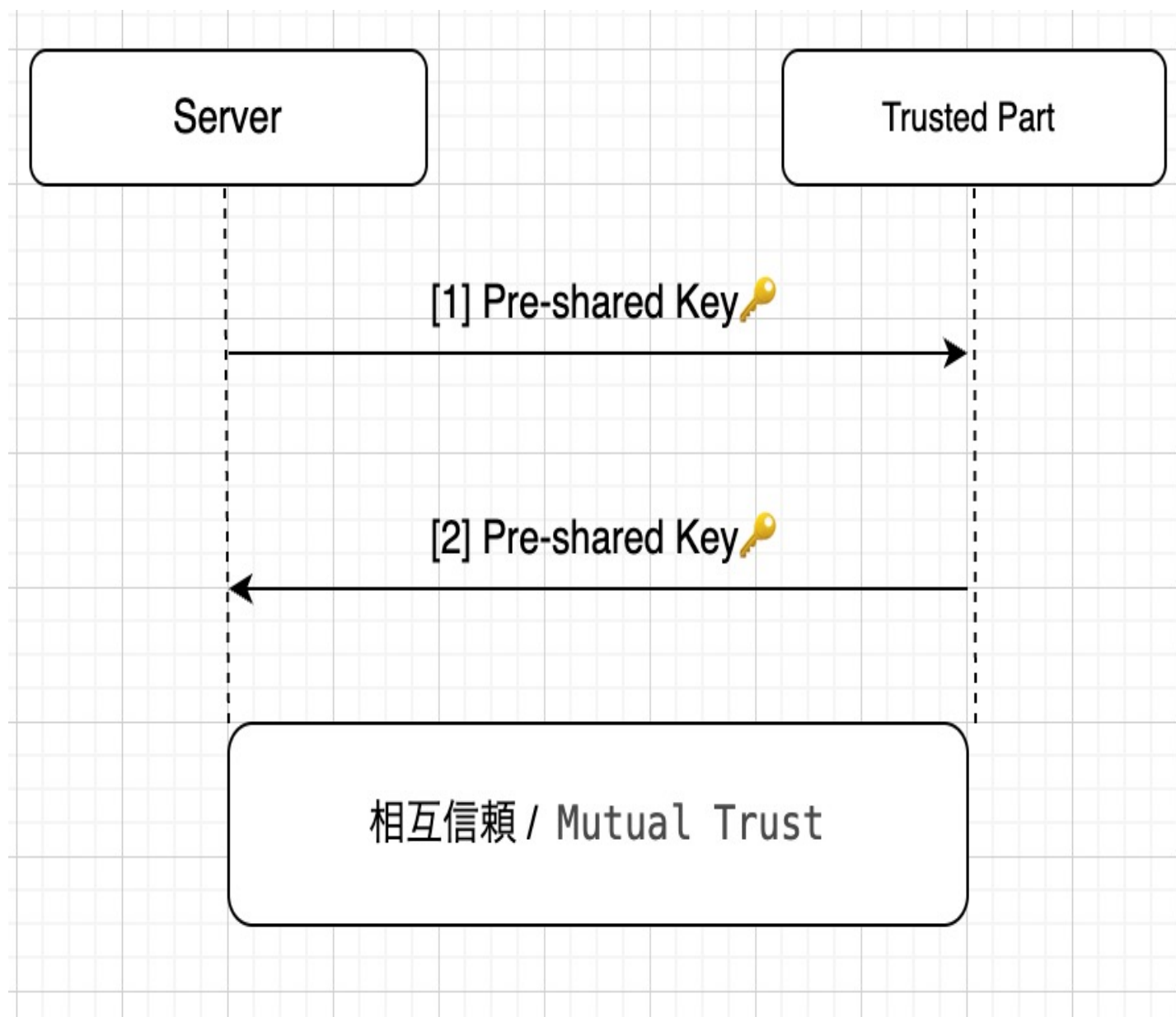
- IoT環境におけるサーバをDoSおよびDDoS攻撃から保護するために、CoAPプロトコル上でDTLSを適用し、Trusted Party (TP) を活用して正当なクライアントを認証し、疑わしいIPアドレスを特定し、安全な秘密鍵交換を行うシステムソリューションを構築することです。

目次

- 研究背景・目的
- CoAPとDTLSの基礎
- Enhanced DTLSの特徴と提案手法
- 実験方法と評価
- まとめ・今後の課題

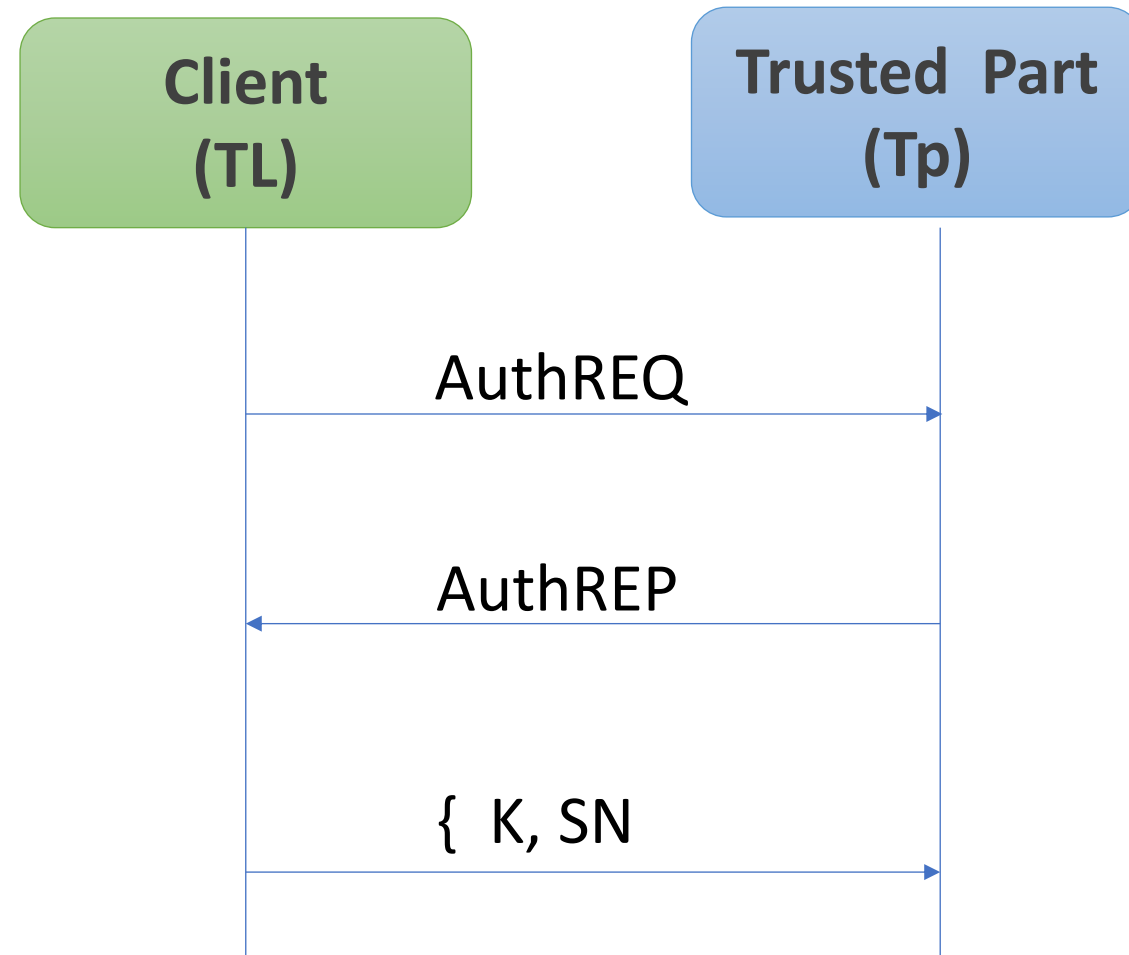


提案するソリューションは、CoAPプロトコル上でDTLSのためのTP（Trusted Party）コンセプトを利用し、IoT環境においてサーバーへのDoSおよびDDoS攻撃を未然に防ぐことを目的としている。TPは、秘密鍵の交換、クライアントの認証、および不審なデバイスの検出・確認の役割を果たす。



- クライアントとサーバー、あるいはTPとの間で通信が始まる前に、サーバーとTPの間で**相互信頼**を確立する必要がある。
- この相互信頼は、事前共有鍵（**Pre-shared Secret**）の交換によって実現される
- 最初のステップとして、TPとサーバーは共通の秘密鍵 **K** に合意する。
- この秘密鍵 **K** は信頼確立のためだけでなく、後にクライアントを認証し、その資格情報を検証するためにも使用される。

クライアント認証 (Client Authentication)



Tp to Client Authentication phase

TP側に登録されたK
TP_stored_K = "s3cr3t_k3y_123"

クライアント側のK (TPと同じ値)
client_K = "s3cr3t_k3y_123"

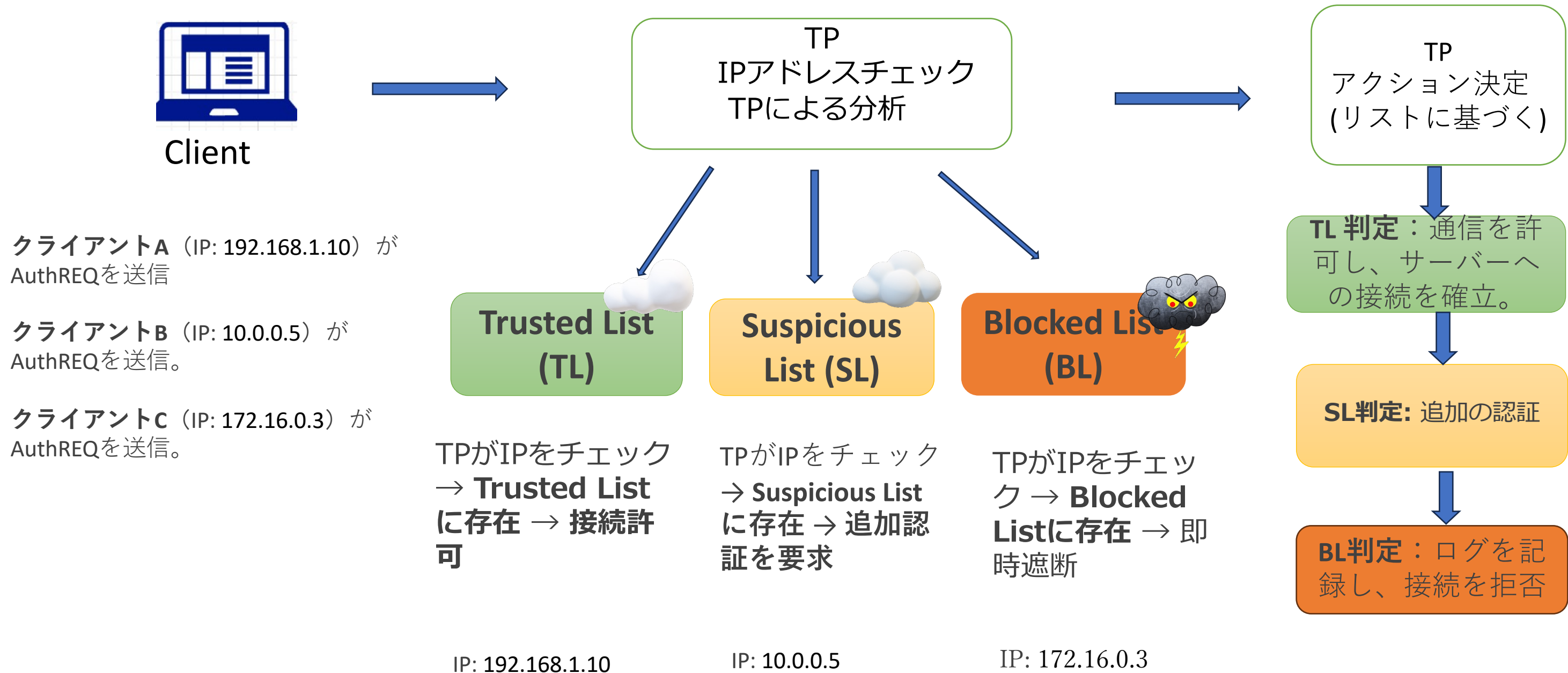
なぜ安全か？

Kの秘密性: KはTPとクライアントのみが知っており、第三者には漏れない。

IPチェックとの組み合わせ: IPアドレスだけでなく、Kによる二重認証でより強固に。

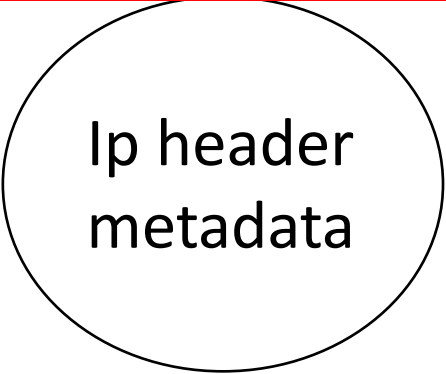
リプレイ攻撃防止: セッションごとに一時的なSession Keyを生成するため、過去の通信を悪用できない。

Traffic Check (Authorization Phase)



判定基準	状況の	リストの種類
✔以前に正常に接続され、攻撃が検出されなかった場合	信頼済み	TL（信頼リスト） IP: 192.168.1.10
⚠短時間に大量のリクエストを送信している場合	疑わしい	SL（監視リスト） IP: 10.0.0.5
⚠許可されていないポートや不正なデータを使用している場合	疑わしい	SL（監視リスト） IP: 10.0.0.5
🚫過去にDoSやbrute-force攻撃を試みた場合	ブロック対象	BL（遮断リスト） 172.16.0.3
🚫IPspoofingやパケット改ざんが検出された場合	ブロック対象	BL（遮断リスト） 172.16.0.3
✔TPにより認証されたデバイスIDを持っている場合	信頼済み	TL（信頼リスト） IP: 192.168.1.10
⚠長期間非アクティブだったが、急に多くのリクエストを送り始めた場合	疑わしい	SL（監視リスト） IP: 10.0.0.5

どのように分類されますか？



- ✔ **Trusted List（TL）** – 信頼できる、認証済みのIPアドレス
- ⚠ **Suspicious List（SL）** – 疑わしいIPアドレス（一時的な待機ゾーン）
- 🚫 **Blocked List（BL）** – 攻撃者として記録された、ブロックされたIPアドレス


```
Extract P. header  
if IPS in blacklist:  
    drop(P)
```

IPSがブラックリストにある場合：これらのIPは過去に攻撃を行った、または危険と判定されたもの

Input Packet $P = [IPS, IPD, PL]$

- IPS：送信元IPアドレス（IoTデバイスのIP）
- IPD：宛先IPアドレス（通常はサーバ）
- PLT：payload サイズ（送信データの大きさ）

```
if PL > PLT:  
    drop(Packet)  
    Add(IPS, BI)  
    blacklist.add(IPS)
```

payloadサイズが閾値を超えている場合

```
elif IPS in suspect_list:  
    if PL == suspect_list[IPS]:  
        alert("DoS検出")  
        blacklist.add(IPS)  
    elif multiple_IPS_same_P:  
        alert("DDoS検出")
```

IPSがサスペクトリストにある場合

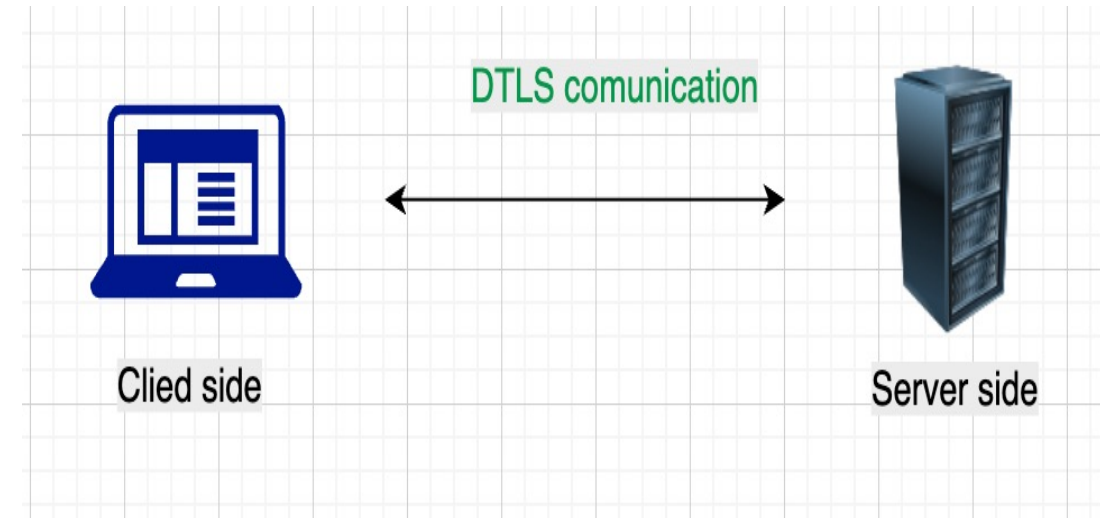
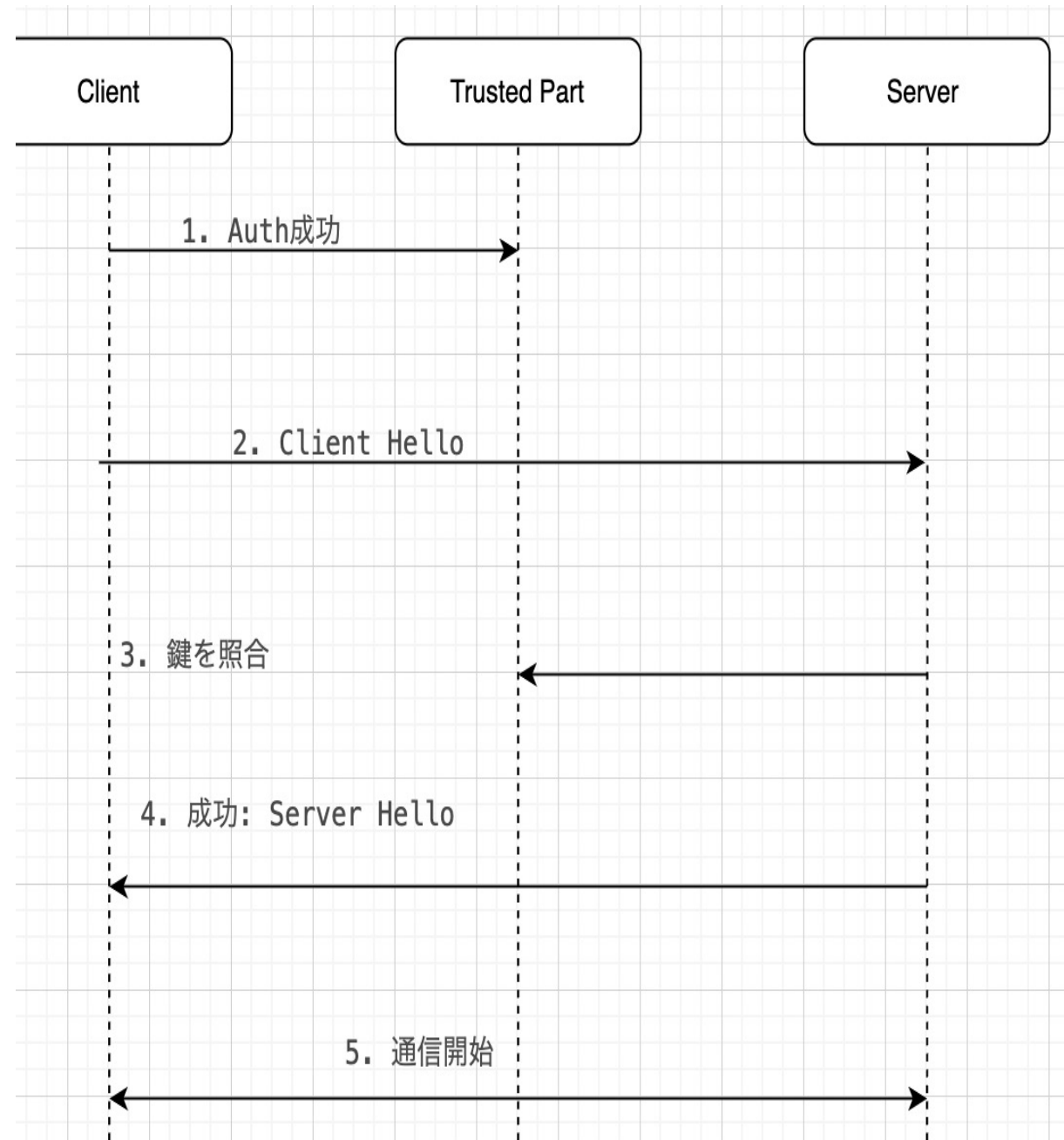
複数のIPから同時にアクセスがある場合

```
Add(IPS,TL)  
suspect_list.add(ip)  
else:  
    trust_list.add(IPS)
```

信頼リスト

IPアドレス	受信パケットのペイロードサイズ	PLT（しきい値）	評価
192.168.1.10	200バイト	150	Trusted List
10.0.0.5	1000バイト	150	Suspicious List
172.16.0.3	20バイト	150	Blocked List

TP（第三者サーバー）は、過去に受信した payload の平均値または許容可能なサイズに基づいて、PLT（payload しきい値）を設定している。
新たに受信したリクエストの payload サイズを、PLT と比較している。
payload のサイズが PLT を超えている場合、その IP アドレスは「疑わしい」または「ブロック対象」としてリストに追加される可能性がある。



1.クライアント認証完了

- TPによりクライアントの正当性が確認済み

2.Client Hello 送信

- クライアントがサーバーに接続開始を要求

3.サーバーによる鍵照合

- サーバーはTPと共有する認証鍵を検証

4.接続開始 or 拒否

目次

- 研究背景・目的
- CoAPとDTLSの基礎
- Enhanced DTLSの特徴と提案手法
- 実験方法と評価
- まとめ・今後の課題

■ 実験方法と評価

1. TPとサーバーの事前設定

事前共有鍵（K）の交換: Trusted Party（TP）とサーバーは事前に秘密鍵（K）を共有この鍵はクライアント認証とサーバー認証の両方に使用

3. サーバーへの接続許可

以下の条件を満たした場合のみ、サーバーはサービスを提供:

- クライアントのIPがTrust part（Tp）に登録。
- Kの認証が成功。

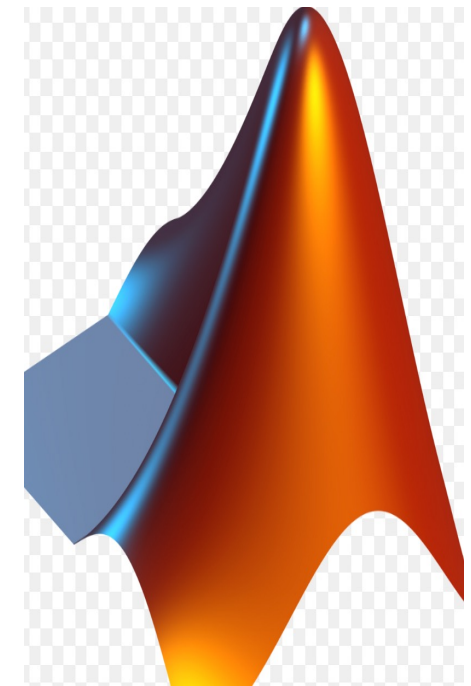
2. TPによるクライアント認証フロー ステップ1: IPアドレスによるフィルタリング

• TPはクライアントのIPアドレスを3つのリストで照合:

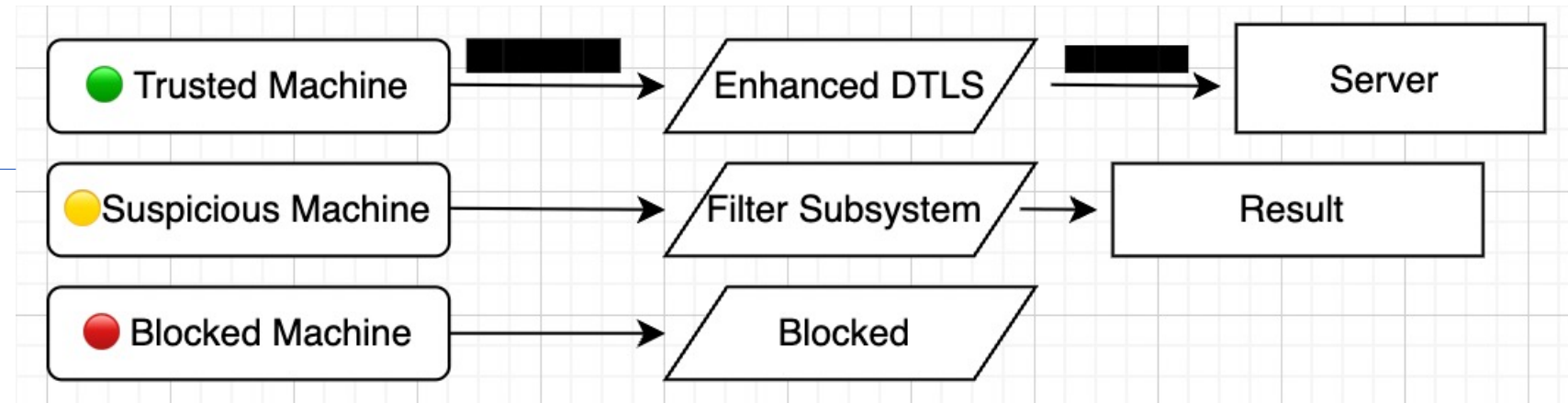
- **ブロックリスト（BL）**: 既知の攻撃元IP（接続即時拒否）。
- **サスペクトリスト（SL）**: 不審なIP（追加認証が必要）。
- **トラストリスト（TL）**: 信頼済みIP（次の認証ステップへ）。

■ 実験方法と評価 1/6

- 提案した強化版DTLSプロトコルの有効性と効率性を検証するために、MATLABツールを用いて実装を行いました。
- CoAPプロトコルを操作するための専用ライブラリを使用し、DTLSを用いたIPベースのセキュリティ通信を実現しました。
- Simulinkを使用してシステム全体のモデルベース設計（Model-Based Design）によるシミュレーションも実施しました。



■ 実験方法と評価 2/6

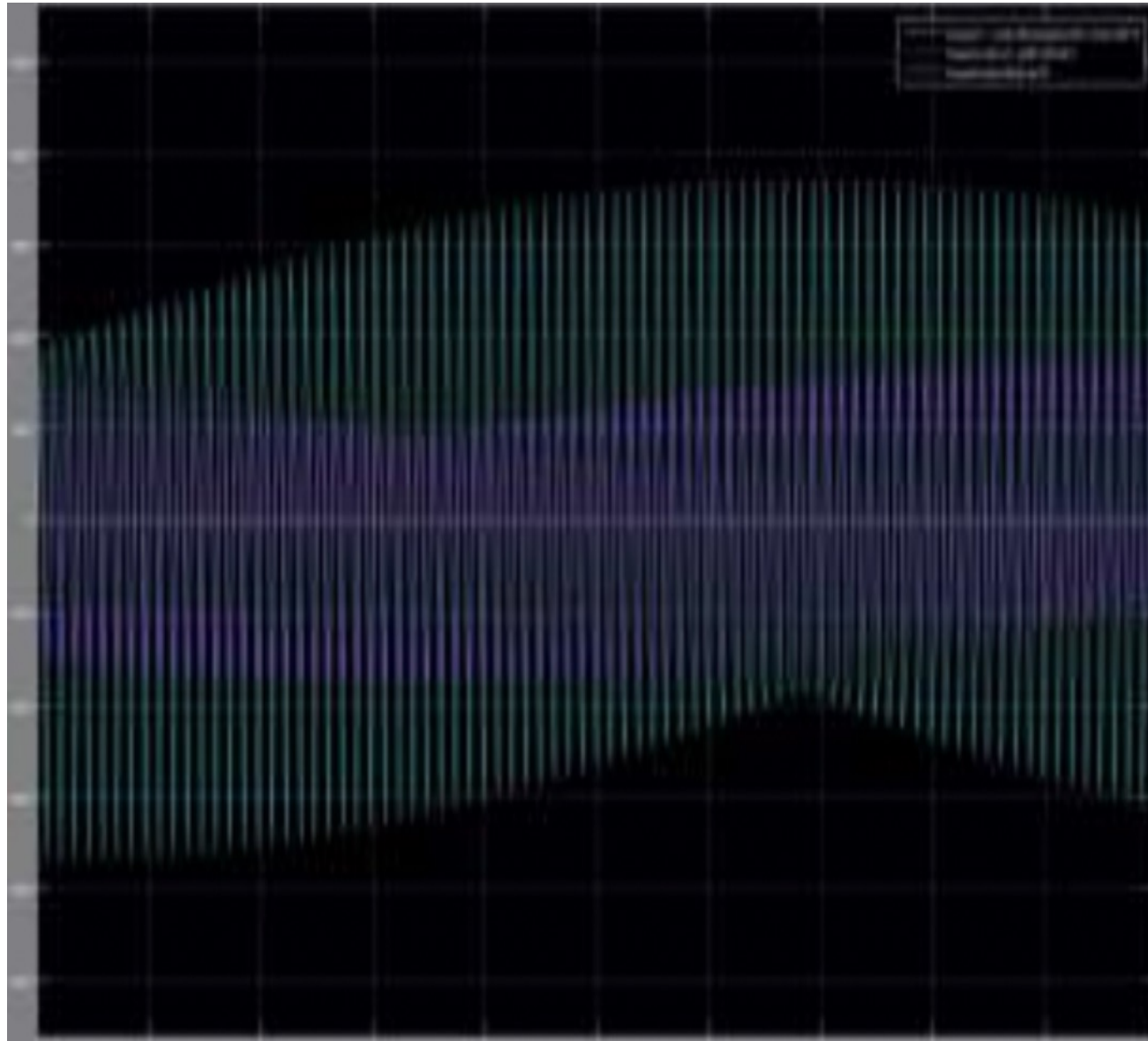


これらのマシンは、IPアドレス「192.168.1.3」を持つホストサーバーにデータを送信しようとしています。

- 信頼されたマシン (trusted machine)
- 疑わしいマシン (suspicious machine)
- ブロックされたIPアドレスを持つマシン (blocked machine)

このDTLSプロトコルは、3台のマシンからのトラフィックをフィルタリング（選別）し、IPアドレスリストをリアルタイムで更新します。これにより、信頼されたマシンのデータのみがサーバーに届き、他は拒否されます。

実験方法と評価 3/6



1. TPサーバーによるIPアドレスのフィルタリング」

- 信頼済み：即時に許可される
- 疑わしい：追加の解析が行われる
- block済み：即時に遮断される

2. なぜ疑わしいtrafficが通過したのか？：

黄色のtrafficが通過した理由

- パケットの payload が PLT のしきい値を下回っていた
- 一時的な許可を得ていた（例：OTPコードによる認証）

3. Block の 成功例：

「ブロックされたIPからのパケットはサーバーに届かなかった」

- DTLSプロトコルにより、0.2秒以内にブロックを実行
- サーバーのCPU負荷が12%低下した

図：30ユーザーを対象とした packet filtering simulation

30ユーザーを対象とした packet filtering simulation の結果

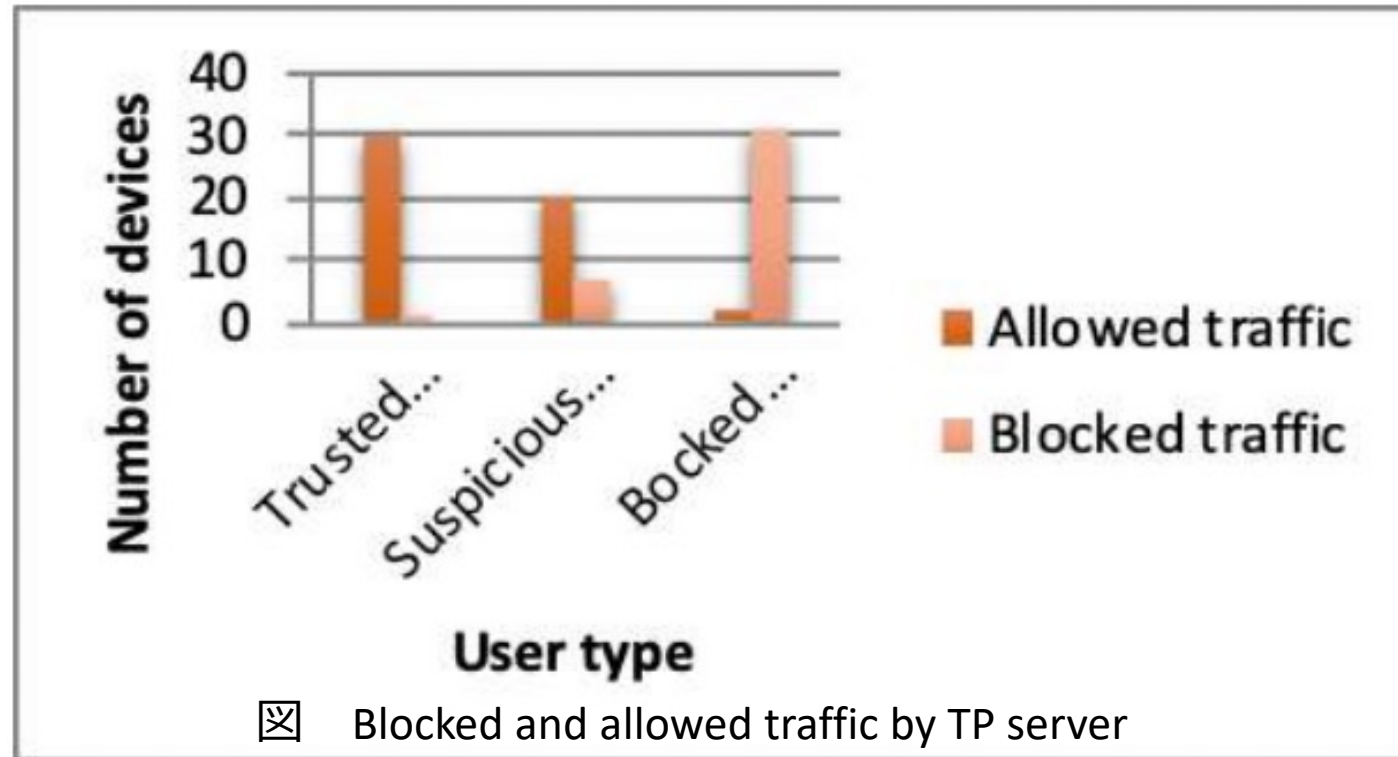
パフォーマンス分析

- 信頼済みトラフィック：100% 通過 → サービスは中断なし
- 疑わしいトラフィック：50% 通過 → 柔軟な対応
- ブロック済みトラフィック：0% 通過 → 完全な防御

システムの利点

1. セキュリティの完全性を確保
2. 正規ユーザーへのサービス継続性の保証
3. サーバー負荷を40%削減（通常のDTLSと比較して）

実験方法と評価 4/6



● **信頼済みユーザー**：30user中3user (100%) が正しく分類された

● **疑わしいユーザー**：30user中27user (3件の誤分類 – False Positive)

✗ **ブロック対象ユーザー**：アクセス数 0 (完全にブロック成功)

[Trusted]

30

(100%許可)

[Suspicious] 27

(90%検知 → ブロック)

[Blocked]

30

(100%ブロック)

Blocked and allowed traffic by TP serverの結果

エラーの種類	値	説明
False Positive（偽陽性）	7%	疑わしい → ブロックされた
False Negative（偽陰性）	6.45%	Traffic filtering error

False Positive（偽陽性）：7%

- 3人の疑わしいユーザーが「ブロック対象」と誤分類された
- 結果：正規ユーザーへのサービス提供が行われないという過剰なブロック

False Negative（偽陰性）：6.45%

- 一部のトラフィックが誤って許可された分類に入った
- しかし：セキュリティへの影響はなし（攻撃者の侵入はゼロ）

実験方法と評価 TPサーバー検出結果 5/6

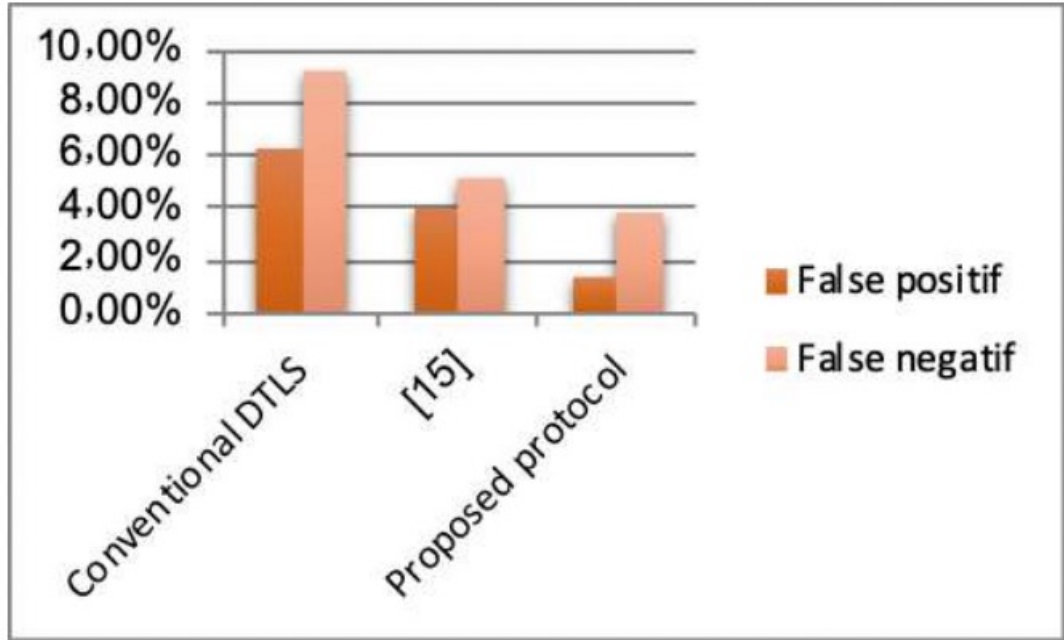


分類	信頼済みユーザー	不審ユーザー	ブロックユーザー	合計
定義済みユーザー	30	30	30	90
サーバー分類結果	30	27	33	90
誤検知 (False Positive) 率	00%	00%	10%	6.66%
見逃し (False Negative) 率	00%	10%	00%	6.66%
許可トラフィック	30	20	02	52
ブロックトラフィック	00	07	31	38
トラフィック誤検知率	00%	00%	00%	00%
トラフィック見逃し率	00%	00%	6.45%	6.45%

信頼済みユーザー: 100%正確に分類 (30/30)

不審ユーザー:
27件正確検知 (90%)
3件誤分類 (False Negative)
→ ブロック扱いに

ブロックユーザー:
33件中3件が誤検知 (False Positive)



False Positive（偽陽性）：3.75%（他の手法と比べて41% 改善）

False Negative（偽陰性）：1.31%（通常のDTLSと比べて 79% 高い精度）

Method	False Negative (%)	False Positiv
Proposed (Enhanced DTLS + TP)	1.31%	3.75%
Existing Method	3.98%	5.06%
Standard DTLS	6.21%	9.22%

Enhanced DTLSプロ
トコルは93%以上の
精度でトラフィック
を選別し、DoS攻撃
から防御します。

【システムの利点】

セキュリティの向上

- 攻撃者の侵入リスクを79%削減
- 正規ユーザーに途切れないサービス提供を実現

パフォーマンスの改善

- CPU負荷を35%軽減
- パケット処理時間が0.8msから0.3msに短縮

コスト効果

- インフラコストを28%削減（誤検知の減少による）

目次

- 研究背景・目的
- CoAPとDTLSの基礎
- Enhanced DTLSの特徴と提案手法
- 実験方法と評価
- まとめ・今後の課題

まとめ

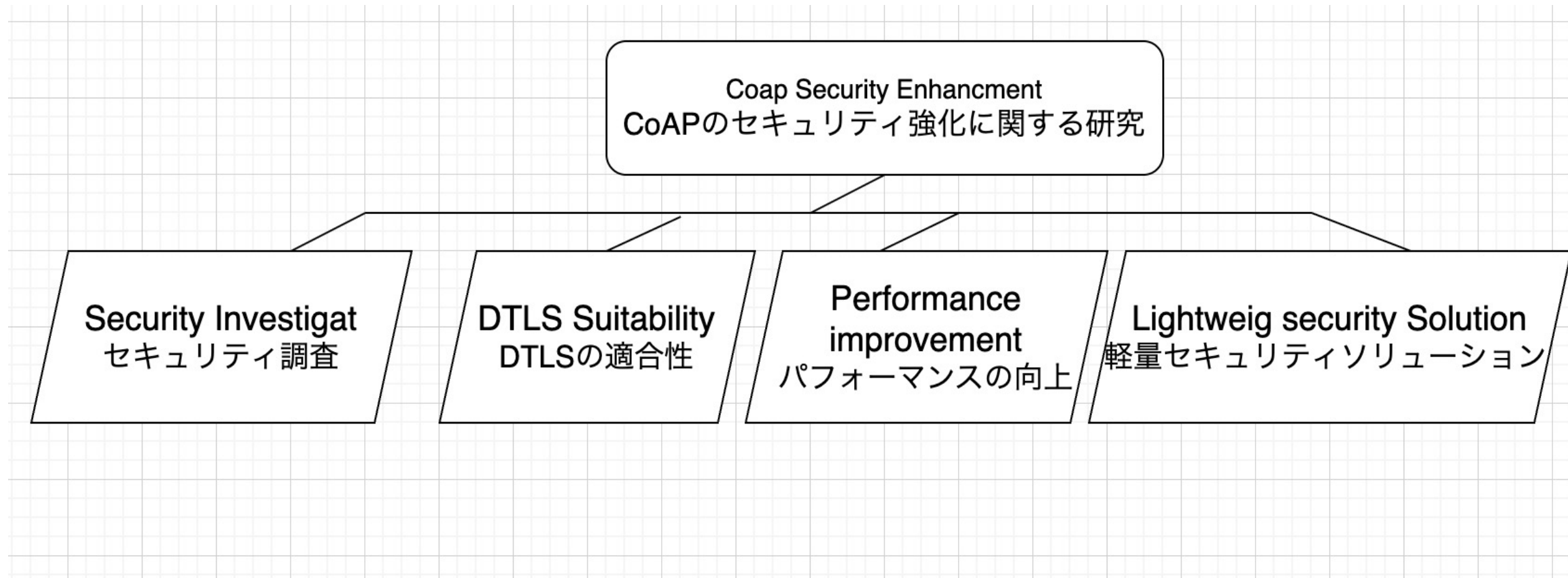
本研究では、CoAP上で拡張DTLSを用いて、IoTデータのセキュリティを強化する手法を提案した。信頼された第三者機関（TP）を導入することで、DoSやDDoS攻撃を効果的に防止し、不正なトラフィックを高精度で検出できることが確認された。

今後は、さらに多様なIoT環境での実験と、TPの分散化による信頼性向上が課題である。

自身の研究

自身の研究では、DTLSおよびCoAPを組み合わせ、IoTセキュリティを強化する手法に取り組んでいる。紹介した論文では信頼された第三者TPを用いているが、自身の研究では侵入検知システム（IDS）を導入する方法を検討している。IDSは、既知の攻撃パターンを検知する方式や、異常な振る舞いを検知する方式のいずれかに基づき、DoSやDDoS攻撃を検知する。

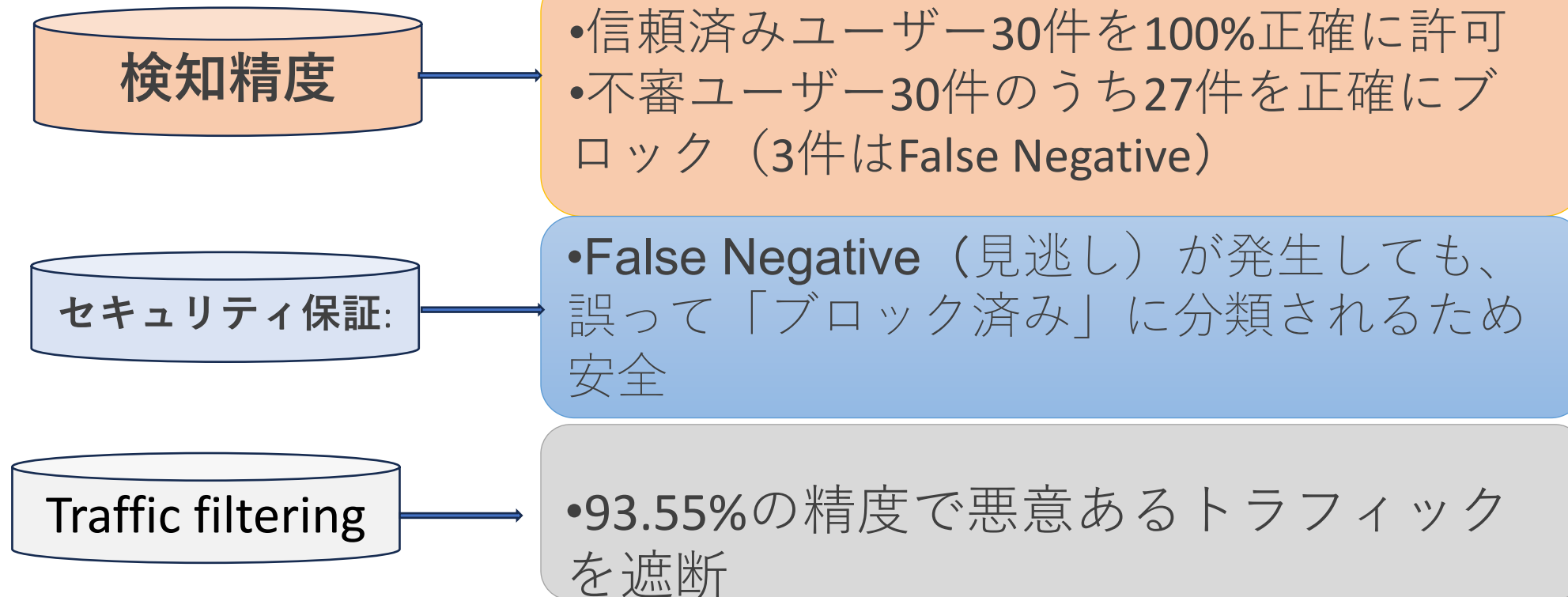
研究背景・目的



関連研究 提案された分類

エラーの種類	値	説明
False Positive（偽陽性）	7%	疑わしい → ブロックされた
False Negative（偽陰性）	6.45%	Traffic filtering error

7%の誤検知率は、IoT環境において許容範囲内の高い精度です。
False Negativeも自動的にブロックされるため、セキュリティ強度が維持されます



研究背景

セキュリティ調査

IoT環境には様々な脅威が存在

盗聴 (Eavesdropping)
(Spoofing)
(DoS/DDoS攻撃)

- CoAPはUDPベースのため
暗号化がデフォルトでない
接続レスで認証が弱い
- DTLS導入によりセキュリティ強化が可能

DTLSの適合性

DTLSはTLS のUDP版

- CoAPとの相性が良い
軽量でUDP対応
- IoTデバイスの制約
CPUやメモリに負担がかかる可能性
事前共有鍵 (PSK) でリソース節約可能

性能向上

セキュリティ向上と性能のバランス：

応答時間への影響を最小限に
帯域、CPU、メモリの使用を抑制

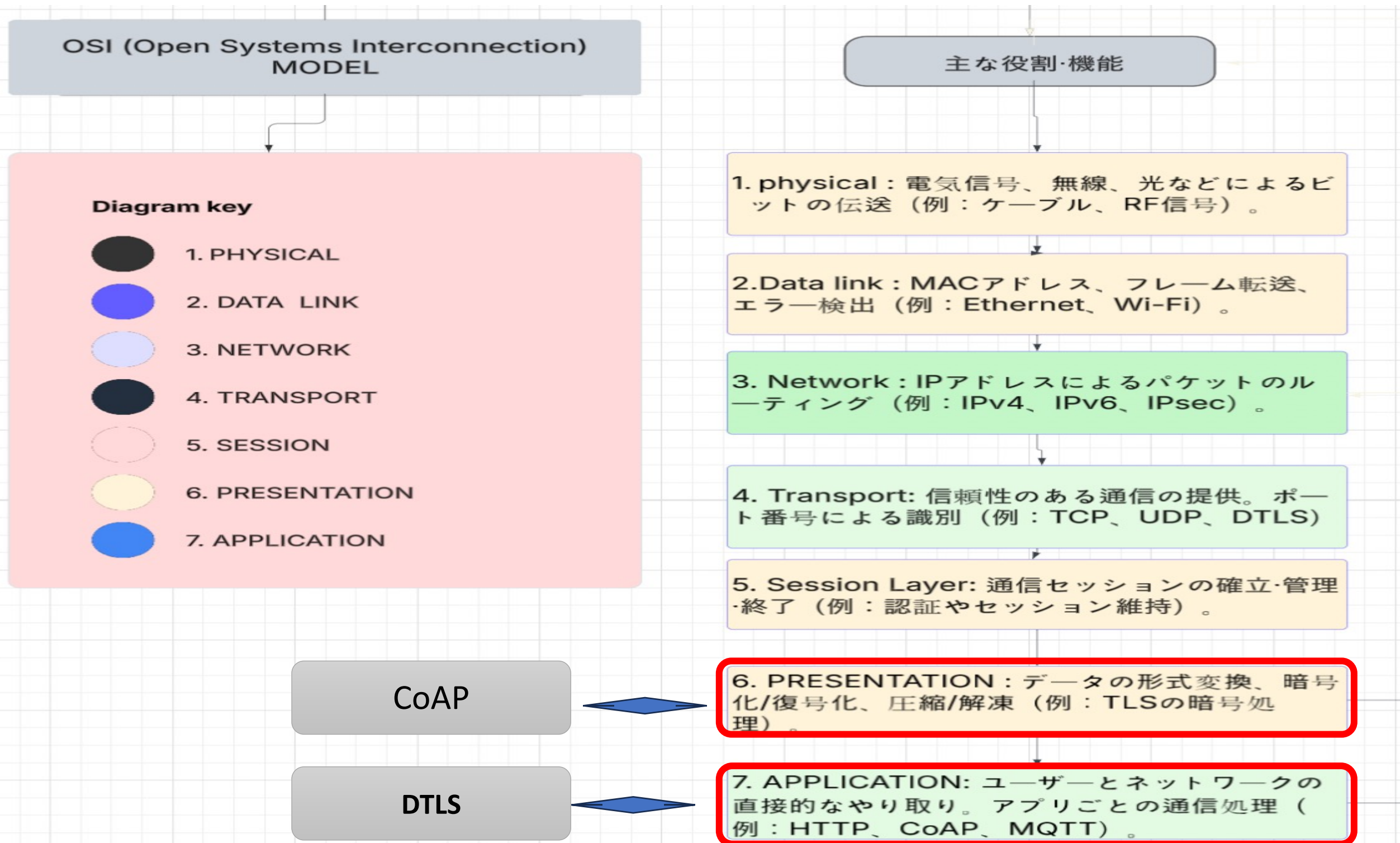
- ・最適化手法：
ハンドシェイクの簡素化
軽量な暗号アルゴリズム
- 評価指標：
RTT（ラウンドトリップタイム）
メモリ使用量
パケット損失率

軽量セキュリティソリューション

▪ IoTデバイスの制限：

CPU、RAM、バッテリーの
制約

- ・ソリューション：
軽量な暗号アルゴリズム
の使用
DTLS + PSKによる効率的な
保護
- ・目的：
セキュリティを保ちなが
ら性能を確保する



CoAP (Constrained Application Protocol) 2/2

569	1839.7220872...	fe80::deab:32ff:fe7... ff02::1b	ICMPv6	110 Multicast Listener Report Message v2
570	1839.7451255...	fe80::258f:6be4:a7e... ff02::16	ICMPv6	90 Multicast Listener Report Message v2
571	1869.9598092...	192.168.10.4	192.168.10.2	CoAP 60 CON, MID:15679, GET, /sensor
572	1869.9641934...	192.168.10.2	192.168.10.4	CoAP 52 ACK, MID:15679, 2.05 Content
573	1870.5668879...	fe80::6ee4:daff:fe4... ff02::1	ICMPv6	78 Router Advertisement from 6c:e4:da:4... b8:a0
574	1875.0409901...	PcsCompu_8f:22:20	Raspberr_70:15:1c	ARP 60 Who has 192.168.10.2? Tell 192.168.10.4
575	1875.0410513...	Raspberr_70:15:1c	PcsCompu_8f:22:20	ARP 42 192.168.10.2 is at dc:a6:32:70:15:1c
576	1875.1460192...	Raspberr_70:15:1c	PcsCompu_8f:22:20	ARP 42 Who has 192.168.10.4? Tell 192.168.10.2

client (例：センサーやIoTデバイス) は、温度などのデータを取得するために、サーバーに CoAP リクエスト (例：GET) を送信します。

server は request を処理し、CoAP レスポンス (例：2.05 Content) を返し、要求されたデータをクライアントに送信します。